Here are the core terminologies in Spring MVC architecture, explained clearly:

---

1. DispatcherServlet (Front Controller)

- The heart of Spring MVC. It's a single servlet that intercepts all incoming HTTP requests and delegates them to the appropriate controllers.
- Acts as a central routing hub, managing the entire request-handling workflow.

---

2. Controller (`@Controller` / `@RestController`)

- A Java class that handles specific HTTP requests (e.g., `/users`, `/products`).
- Contains methods mapped to URLs using `@RequestMapping`, `@GetMapping`, `@PostMapping`, etc.
- Processes user input, calls services, and returns a Model + View name (or JSON data if `@RestController`).

---

3. Model (`Model`, `ModelMap`, `ModelAndView`)

- Holds the application data (business objects) that will be displayed in the view.
- The controller adds data to the `Model` object, which is then passed to the view layer (e.g., JSP, Thymeleaf).

---

4. ViewResolver (e.g., `InternalResourceViewResolver`)

- A strategy interface that resolves logical view names (like `"home"`) to actual view templates (like `/WEB-INF/views/home.jsp`).
- Decouples controller logic from specific view technologies.

---

5. View (JSP, Thymeleaf, Freemarker, etc.)

- The presentation layer that renders the final response (HTML, JSON, XML).
- Uses the data from the `Model` to dynamically generate the output sent back to the client.

---

6. HandlerMapping

- A component that maps incoming request URLs to the appropriate controller methods (handlers).
- Example: `@RequestMapping("/user")` maps to the `UserController`'s method.

---

7. HandlerAdapter

- Executes the handler (controller method) found by `HandlerMapping`.
- Converts the request data (parameters, body) into method arguments and handles return values (ModelAndView or JSON).

---

8. ModelAndView

- A container object used in older Spring MVC that holds both the `Model` data and the logical `View` name to be rendered.

---

9. Interceptor (`HandlerInterceptor`)

- Similar to a filter but more Spring-aware. Allows pre-processing (before controller), post-processing (after controller but before view), and completion (after view) logic — useful for logging, authentication, etc.

---

10. Form-Backing Object / Command Object

- A plain Java object (POJO) used to bind HTML form fields automatically to Java properties via data binding.

---

11. Validator (`Validator` interface / `@Valid` + `BindingResult`)

- Used to validate user input. Spring integrates with Bean Validation (e.g., Hibernate Validator) using annotations like `@NotNull`, `@Size`, etc.

Quick Flow Example:

1. Client hits `/login` → DispatcherServlet
2. HandlerMapping finds the controller method mapped to `/login`
3. HandlerAdapter invokes that method
4. Controller processes logic, adds data to Model, returns logical view `"loginSuccess"`
5. ViewResolver resolves it to `/WEB-INF/views/loginSuccess.jsp`
6. View renders the final HTML with model data → response sent back.

This separation of concerns makes Spring MVC highly modular, testable, and maintainable.